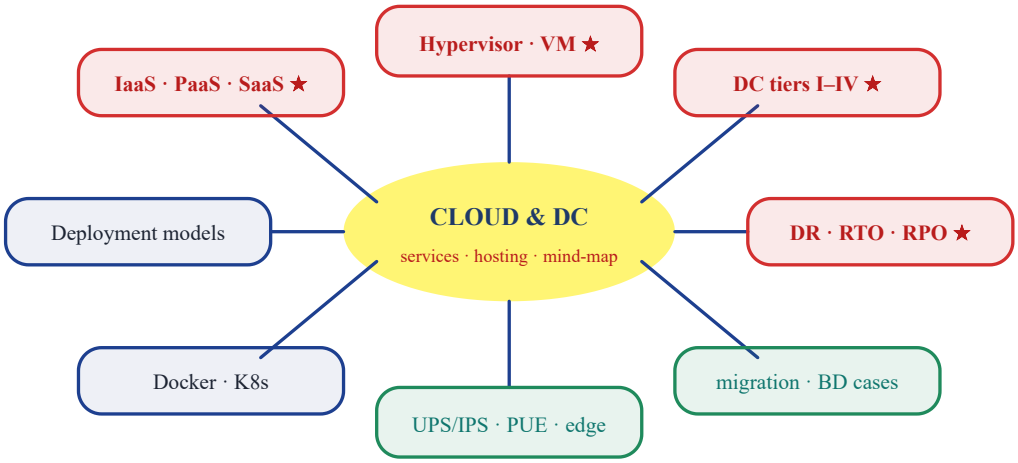
Cloud Computing, Virtualisation & Data Centres

IaaS/PaaS/SaaS · deployment models · hypervisors & containers · tiers · UPS/IPS · DR, RTO/RPO · edge · migration: exam-ready in 15 sheets



↑ red = high-yield, blue = core, green = applied / Bangladesh

Syllabus coverage

"Introduction to Computer Systems: … **Basic Information Technology**; **The Internet** …". Written bank papers stretch this line to cloud computing, virtual machines, data centres, disaster recovery and cloud migration. Networks, DNS and cyber attacks have their own notes.

High-yield list

① NIST definition, 5 traits
② IaaS vs PaaS vs SaaS
③ public/private/hybrid
④ Type-1 vs Type-2
⑤ VM vs container
⑥ Tier II vs Tier III
⑦ UPS vs IPS
⑧ RTO vs RPO, DC vs DRS

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

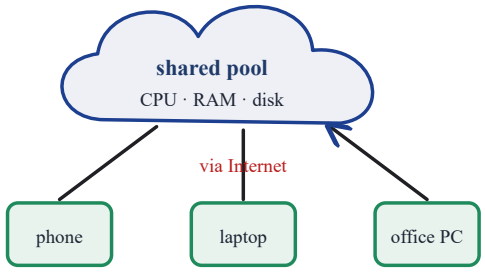
✓ Cloud computing: NIST definition … p. 2
✓ Service models IaaS/PaaS/SaaS … p. 3
✓ Deployment models …… p. 4
✓ Cloud security & Bangladesh … p. 5
✓ Virtualisation & hypervisors … p. 6
✓ Containers, Kubernetes, scaling … p. 7
✓ Data centres & tiers …… p. 8
✓ Power: UPS vs IPS, redundancy … p. 9
✓ Cooling, PUE, servers …… p. 10
✓ DR, RTO/RPO, maintenance … p. 11
✓ Edge, fog, CDN & big data … p. 12
✓ Migration & scenarios …… p. 13
✓ Night-before sheet …… p. 14
✓ Practice MCQs + written …… p. 15

1 What Is Cloud Computing?

NIST SP 800-145 (2011): Cloud computing is a model for enabling ubiquitous, convenient, **on-demand network access to a shared pool of configurable computing resources** (networks, servers, storage, applications, services) that can be **rapidly provisioned and released** with minimal management effort or service-provider interaction. NIST counts **5 characteristics · 3 service models · 4 deployment models**.

The 5 essential characteristics

1. **On-demand self-service**: start a server from a portal, no human at the provider
2. **Broad network access**: reached over the network from phones, laptops, PCs
3. **Resource pooling**: one pool serves many tenants (**multi-tenancy**); location-independent
4. **Rapid elasticity**: scale out and back in, seemingly unlimited
5. **Measured service**: metered, **pay-as-you-go** like electricity



Cloud vs traditional (on-premises) computing

Point	Traditional / on-premises	Cloud
Cost model	**CapEx**: buy servers up front	**OpEx**: pay per use, monthly bill
Provisioning	weeks (purchase, install)	minutes, self-service
Scaling	sized for the **peak**; idle most of the year	elastic: grows and shrinks with load
Control & data location	full control, data on site	less control; data in provider's region

Advantages: low up-front cost, elasticity, high availability, global reach in minutes, automatic updates, built-in backup and DR, provider maintains hardware.

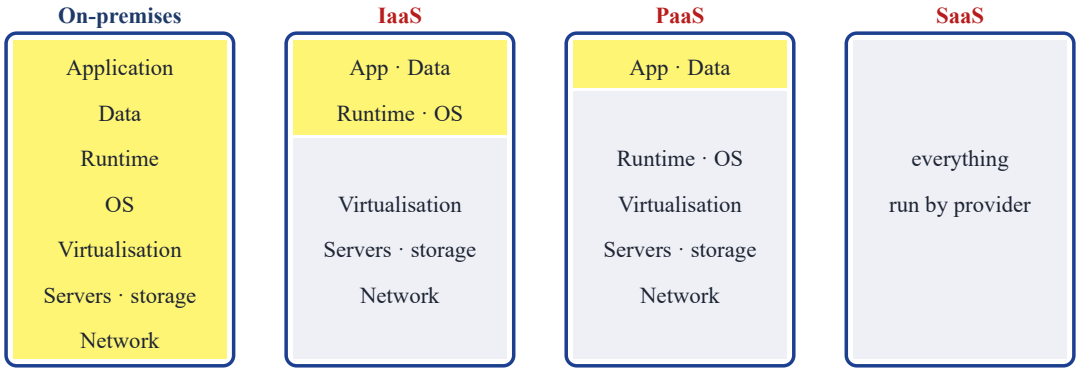
Disadvantages / challenges: needs reliable Internet, **vendor lock-in**, data sovereignty, provider outages, egress costs. **Solved by** multi-region design, own-key encryption, exit plan, SLA penalties, cost monitoring.

Memory trick: 5 traits = "**O-B-R-R-M**": **O**n-demand, **B**road access, **R**esource pooling, **R**apid elasticity, **M**easured. "Cloud is a **utility**: switch on, use, pay the meter."

Exam trap: The concept tied to **pooling and sharing** of resources is **virtualisation**. A cloud service is usually **stateless** (each request carries what it needs), which is why it scales out easily.

2 Service Models: IaaS, PaaS, SaaS

Rule: The service models differ in **how much of the stack the provider manages**. The higher you go (IaaS → PaaS → SaaS), the less you manage and the less you control.



yellow = you manage · blue = provider manages

Model	You get	You manage	Typical user	Examples
IaaS	virtual servers, disks, networks	OS, runtime, app, data	system / network admin	AWS EC2, Azure VMs, Google Compute Engine
PaaS	a ready platform to **deploy code**	only app + data	developer	Google App Engine, Heroku, AWS Elastic Beanstalk, Azure App Service
SaaS	a finished application in a browser	only settings, users	end user	Gmail, Microsoft 365, Salesforce, Zoom, Dropbox

Worked scenario: A startup wants to write and deploy code only, with **no hardware, OS or runtime** to manage → **PaaS**. Examples: Google App Engine, Heroku (or AWS Elastic Beanstalk, Azure App Service). IaaS would still leave the OS to them; SaaS gives no place for their own code.

Memory trick: **Pizza as a service**: make at home = on-prem; take-and-bake = IaaS; delivery = PaaS; dine in a restaurant = SaaS. "**I**nfrastructure for **I**T admins, **P**latform for **P**rogrammers, **S**oftware for **S**taff."

Other "as a Service"

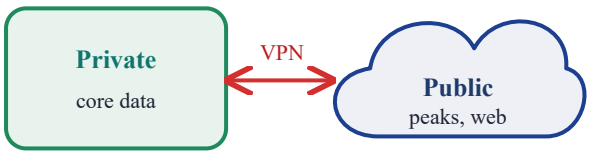
- **FaaS / serverless**: run a function per event (AWS Lambda, 2014; Azure Functions)
- **DBaaS**: managed database (Amazon RDS, Azure SQL); the "cloud-based database"
- **STaaS**: storage (Amazon S3, 2006; Google Drive)
- **DaaS**: virtual desktops; **CaaS**: containers as a service

Exam trap: The 3 NIST "layers" are IaaS, PaaS, SaaS; "Computing as a Service" is **not** one of them. The 3 basic cloud functions are **compute, storage and networking**. Security in the cloud is **shared**: provider secures the cloud, customer secures what it puts in it.

3 Deployment Models & Enabling Technologies

Deployment model: says **who owns and who may use** the cloud. NIST lists four: **public, private, community, hybrid**. Industry adds **multi-cloud** (two or more public providers).

Model	Used by	Strength	Weakness	Example
Public	anyone, over the Internet	cheapest, huge scale	least control, shared tenancy	AWS, Azure, Google Cloud
Private	**one** organisation (on site or hosted)	control, compliance	costly, limited elasticity	bank's own OpenStack / VMware cloud
Community	a group with shared concerns	costs shared, common rules	limited members, governance	a government cloud for ministries
Hybrid	private + public, linked	sensitive data private, peaks public (**cloud bursting**)	complex integration, security at the joins	core banking on-prem, website on public cloud



Hybrid = both, one managed whole

5 key enabling technologies

1. **Virtualisation**: one physical host, many VMs
2. **Broadband networks & data centres** at scale
3. **Service-oriented architecture**, web services and REST APIs
4. **Multi-tenancy & automation** (orchestration, self-service portals)
5. **Utility & grid computing** ideas: metered, distributed resources

Milestones: Salesforce SaaS (1999) · **AWS S3 & EC2 (2006)** · Google App Engine (2008) · Microsoft Azure (2010).

Viva: may a Bangladeshi bank use the cloud?: "Yes, under the Bangladesh Bank cloud guideline (2023, p. 5). Benefits for a bank: lower cost, fast scaling for festival-time transactions, DR in another region. Drawbacks: data sovereignty, dependence on the provider and the link, regulatory approval, skills gap. Keep core banking and customer data in-country (private/hybrid); move websites, email, analytics and DR first."

Memory trick: "**P**ublic = **P**ark (anyone), **P**rivate = home, **C**ommunity = **C**lub, **H**ybrid = **H**ome + park." Hybrid's signature word: **cloud bursting**.

Exam trap: Private cloud is defined by **exclusive use**, not location: a provider can host it off site. Hybrid needs the two parts to work together (data and app portability); owning two separate clouds is just multi-cloud.

4 Cloud Security, Audit & the Bangladesh Rules

Shared responsibility model: the provider secures **"the cloud"** (buildings, hardware, hypervisor, its network); the customer secures **"what it puts in the cloud"** (data, identities, configuration, its code). The split moves with the service model.

Layer	IaaS	PaaS	SaaS
Data, users, access rights	customer	customer	**customer**
Application code	customer	customer	provider
OS, patches, runtime	customer	provider	provider
Hypervisor, hardware, facility	provider	provider	provider

Top cloud threats

- **Misconfiguration** (public storage bucket, open ports): the most common cause of cloud breaches
- weak identity and access management, no MFA, stolen keys → **account hijacking**
- insecure APIs, data breach and loss, insider misuse

Security assessment & audit: ① **vulnerability scans** and **penetration tests** find weaknesses ② **CSPM** (cloud security posture management) tools check every setting against a baseline (CIS benchmarks) and flag drift ③ **audit reports** compare controls with a standard: **ISO/IEC 27001**, 27017 (cloud), 27018 (personal data), SOC 2, PCI DSS v4.0 (2022) ④ findings → risk rating → fix → re-test → continuous monitoring (SIEM, logs).

Cloud in Bangladesh: may a bank use it?

Bangladesh: **Bangladesh Bank Guidelines on Cloud Computing** (BRPD Circular No. 05, **16 March 2023**) apply to banks, NBFIs, MFS providers and payment system operators; compliance was due by **December 2023**. So **yes, financial institutions may use cloud**, under a risk-based framework: a board-approved cloud policy, risk assessment and due diligence on the provider, an SLA with exit plan, encryption and access control, audit and regulator access, and care with customer data (the Bank Company Act 1991, s.12, bars moving records abroad without Bangladesh Bank's permission). Government agencies follow **GoBISM 2016** (BCC's information security manual).

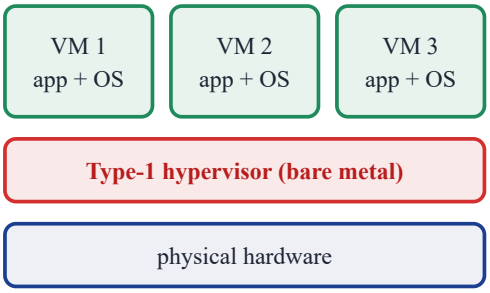
Securing an e-banking web server in our DC: web server in a **DMZ** behind a next-generation firewall and **WAF**; IPS; TLS 1.2/1.3 certificates and HSTS; load balancer; hardened, patched OS; MFA for admins; database in an inner zone; HSM for keys; SIEM logging; regular VAPT; backups and a DR copy.

Memory trick: "Provider guards the **house**; you lock your own **cupboard**." Audit cycle: "**Scan, Score, Fix, Re-check**."

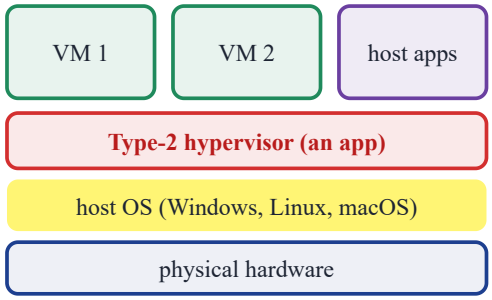
Exam trap: Even in SaaS the customer still owns **data and user access**; a leaked password is never the provider's fault. An ISO 27001 certificate shows a managed process, not that a system cannot be breached.

5 Virtualisation, VMs & Hypervisors

Virtualisation: creates a **software version of hardware** (CPU, memory, disk, network) so one physical machine runs several isolated **virtual machines**. A **VM** is a complete emulated computer with its own guest OS, virtual CPU, RAM, disk and NIC. The **hypervisor** (virtual machine monitor, VMM) creates, runs and isolates VMs and shares out the real hardware.



servers, data centres



desktops, labs, testing

Point	Type-1 (native, bare-metal)	Type-2 (hosted)
Runs on	directly on hardware	on top of a host OS
Speed, security	**faster**, smaller attack surface	slower (extra layer); host OS bugs matter
Examples	**VMware ESXi, Microsoft Hyper-V, Xen, KVM**	**Oracle VirtualBox, VMware Workstation, Parallels**

Kinds of virtualisation

- **Full**: guest OS unmodified; hypervisor traps privileged instructions (binary translation)
- **Para-virtualisation**: guest OS modified to call the hypervisor (hypercalls); early Xen
- **Hardware-assisted**: CPU support, Intel VT-x / AMD-V (2005–06)
- Also: storage (SAN, vSAN), network (VLAN, SDN), desktop (VDI), application

Benefits of a VM: server consolidation (fewer boxes, less power), isolation (one crash stays inside), snapshots and quick rollback, **live migration** (vMotion) with no downtime, fast provisioning from templates, easy DR (a VM is a set of files), run old OSs, safe testing sandbox. **Cost**: hypervisor overhead, licence cost, one host failure hits all its VMs.

Worked example: VM sizing: Host: 32 cores, 96 GB RAM, 4 TB. Each VM: 4 cores, 16 GB, 500 GB. CPU → 32/4 = 8; RAM → 96/16 = **6**; disk → 4000/500 = 8. Max VMs without overcommit = **min(8, 6, 8) = 6**; **RAM** is the limiting resource (8 cores and 1 TB stay free).

Memory trick: Type-**1** = **1**st layer on metal; Type-**2** = **2**nd, needs a host. "**E**SXi, **H**yper-V, **X**en, **K**VM: **EH, X K**ings are bare."

Exam trap: The **Java Virtual Machine** is a process VM: it runs bytecode and manages memory; it is **not** a hypervisor. Hyper-V and KVM count as Type-1 even though they ship inside Windows/Linux.

6 Containers, Docker & Kubernetes

Container: packages an application with its libraries and settings and runs it as an **isolated process sharing the host OS kernel** (namespaces + cgroups). **Docker** (2013) builds and runs containers; **Docker Hub** is its **cloud image registry**. **Kubernetes** (K8s, open-sourced by Google 2014, now CNCF) orchestrates many containers across many machines.

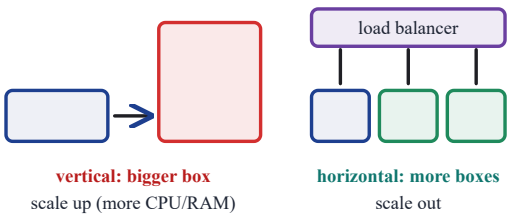
Point	Virtual machine	Container
What is virtualised	hardware; **each VM has a full guest OS**	the OS; all containers **share one kernel**
Size, start time	GBs; boots in minutes	**MBs; starts in seconds**
Resource use, density	heavier; tens per host	light; hundreds per host
Isolation	**stronger** (separate kernels)	weaker; a kernel flaw affects all
Pick when	different OSs, legacy apps, strict isolation	microservices, CI/CD, same image dev → prod

Kubernetes architecture

- **Control plane**: **kube-apiserver** (front door), **etcd** (key-value store of cluster state), **kube-scheduler** (places pods on nodes), **kube-controller-manager** (keeps desired state), cloud-controller-manager
- **Worker node**: **kubelet** (agent, runs pods), **kube-proxy** (service networking), container runtime (containerd, CRI-O)
- Objects: **Pod** (smallest unit), Deployment, Service, Ingress

VM or container for data transfer?: For a reliable, efficient transmission service choose **containers**: lighter, start in seconds, scale out under K8s with health checks and auto-restart. Run them **inside VMs** where strong isolation is needed; the answer is often "both".

7 Scaling: Vertical, Horizontal, Auto



- **Vertical**: simple; limited by the biggest box, downtime, single point of failure
- **Horizontal**: near-unlimited, fault tolerant; app must be **stateless** or share session state
- **Auto scaling**: a metric (CPU > 70%) triggers a policy that adds instances behind the load balancer and removes them when load falls (min/max counts)

Memory trick: "**VM brings its own OS, a container borrows the kernel.**" Vertical = **taller** server, horizontal = **wider** rack. K8s control plane = "**A E S C**": API server, etcd, scheduler, controller manager.

8 Data Centre: Components & Tiers

Data centre: a secured facility that houses an organisation's **servers, storage and network equipment** with conditioned power, cooling, fire protection, physical security and monitoring, so that IT services run 24/7. A **server** is a computer that provides services to clients (web, database, mail, DNS, file); servers use ECC RAM, redundant PSUs and hot-swap disks, so they are more reliable than PCs.

Components (hardware required)

- **Compute**: rack and blade servers in 42U racks
- **Storage**: SAN / NAS, disk arrays (SAS/SSD), tape backup
- **Network**: core/distribution/access switches, routers, firewalls, load balancers, structured cabling, carrier links
- **Power**: utility feeds, transformer, ATS, **UPS**, batteries, generator, PDUs
- **Cooling**: precision AC (CRAC/CRAH), raised floor, hot/cold aisles
- **Security, fire, monitoring**: access control, CCTV, gas suppression, DCIM/NOC (p. 10)

Uptime Institute tiers (TIA-942 uses Rated 1–4)

Tier	Name	Power/cooling paths	Redundancy	Availability	Downtime/yr
I	Basic capacity	1 path	N	99.671%	28.8 h
II	Redundant capacity components	1 path	N+1 components	99.741%	22 h
III	Concurrently maintainable	1 active + 1 alternate	N+1	**99.982%**	1.6 h
IV	Fault tolerant	2 active, compartmentalised	2N / 2(N+1)	99.995%	26.3 min

Tier II vs Tier III: Both have spare (N+1) components, but Tier II has **one distribution path**, so maintaining that path stops the IT load. Tier III adds a **second path**: any component or path can be taken down for maintenance **without shutting down** (concurrently maintainable). Bank primary DCs are normally Tier III or better.

Bangladesh: The **national Tier IV data centre** at Bangabandhu Hi-Tech City, **Kaliakoir, Gazipur**, run by Bangladesh Data Center Company Ltd (BDCCL), was inaugurated on **28 Nov 2019**, Uptime Tier IV certified, 2N+1, 99.995%. BCC's **National Data Centre** (Agargaon, Dhaka) hosts government e-services and the government cloud.

Worked example: series vs parallel reliability: PSU, motherboard, storage in **series**, each $R = 0.95 \rightarrow R = 0.95^3 =$ **0.857** (85.7%). In **parallel** two 0.95 PSUs $\rightarrow 1 - 0.05^2 = 0.9975$: that is why redundancy works.

Memory trick: Tier downtime "**29, 22, 1.6, 0.4**" hours. Tier III = **"maintain without stopping"**; Tier IV = **"fail without stopping"**.

Exam trap: Higher tier ≠ "more servers"; tiers rate **site infrastructure** (power, cooling paths), not IT equipment. Uptime uses Roman tiers I–IV; TIA-942 uses "Rated 1–4".

9 Power: UPS vs IPS, Generators

UPS: (Uninterruptible Power Supply) keeps power flowing with **zero or a few milliseconds' break** and conditions it (surge, sag, noise). It bridges the gap until the generator starts. An **online (double-conversion)** UPS always feeds the load from the inverter: **0 ms transfer**.

IPS: (Instant Power Supply) is an inverter-and-battery backup common in Bangladeshi homes and offices for **lights and fans**. It switches over only after mains fails, with a noticeable delay (tens of ms) and often unclean output: computers may reboot.

Point	UPS (online)	IPS
Changeover	**0 ms** (online) / 2–10 ms (offline)	delayed switch after outage
Output quality	pure sine, regulated, filtered	may be modified sine, unregulated
Backup time	minutes (bridge to generator)	hours (big batteries)
Used for	**servers, network, data centre**	lights, fans, ordinary loads

UPS types: offline/standby (cheapest, PCs) → line-interactive (AVR, small racks) → **online double-conversion** (data centres). Power chain: utility → ATS → UPS → PDU → dual PSUs in each server; **ATS** (automatic transfer switch) brings in the generator within ~10–15 s.

Which generator for a DC?: **Diesel** standby generator: starts and takes load in seconds, fuel stored on site (24–72 h tank), proven; higher running cost and emissions. **Natural-gas**: cheaper fuel and cleaner, but depends on pipeline pressure (gas shortage in BD) and starts slower; good for prime power. **Choice**: diesel for standby/emergency at a bank DC, sized N+1 with AMF panel, fuel contract and monthly load test; gas only if supply is assured. ("DC generator" in such questions means the data-centre generator; standby gensets give AC.)

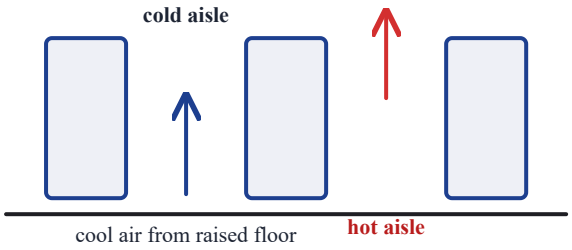
Redundancy words (power and cooling)

- **N**: just enough capacity; no spare
- **N+1**: one spare unit (e.g. 4 UPS modules where 3 are needed)
- **2N**: a complete second, independent system (A and B feeds)
- **2(N+1)** / 2N+1: two systems, each with its own spare
- **Concurrently maintainable**: any part can be serviced without stopping the IT load (Tier III)
- **Fault tolerant**: survives any single unplanned failure (Tier IV)
- **Dual-corded** servers: two PSUs on two PDUs, so one feed can fail

Memory trick: "**U**PS = **U**nbroken; **I**PS = **I**nterrupts a little." Chain order: "**U-A-U-P-S**" = Utility, ATS, UPS, PDU, Server.

Exam trap: A UPS is **not** long-term backup: its battery lasts minutes; the generator carries the load. N+1 ≠ 2N: N+1 adds one unit, 2N doubles the whole system.

10 Cooling, PUE & Energy Efficiency



Why cooling? Every watt the IT uses becomes heat; above the safe range (ASHRAE recommends about **18–27 °C** inlet) chips throttle, fail early, and the room can overheat within minutes. Methods: precision CRAC/CRAH units, **hot/cold aisle containment**, blanking panels, free (outside-air) cooling, in-row and liquid cooling for dense AI racks.

PUE: **PUE = total power ÷ IT power** (total facility power ÷ IT equipment power). Ideal 1.0; global average about **1.56** (Uptime survey 2024). Example: 1,500 kW total, 1,000 kW IT → PUE 1.5.

Challenges in data-centre energy efficiency: cooling takes a large share of power · old, inefficient equipment and low server utilisation (idle servers still draw ~half power) · rising rack density from AI/GPUs · hot climate and humidity (Bangladesh) limit free cooling · costly retrofits, legacy layouts · unreliable grid needing generators · measuring accurately (no metering) · balancing redundancy (2N) against efficiency. **Fixes**: virtualise and consolidate, containment, raise set points, efficient UPS (eco mode), VFD fans, renewables, DCIM monitoring.

Physical security & fire

- layered access: fence, guards, **biometric** + card, **mantrap**, locked cages and racks
- CCTV with recording, visitor log, escort rule
- very early smoke detection (VESDA); **clean-agent gas** suppression (FM-200, Novec 1230), never water sprinklers on live racks
- water-leak, temperature and humidity sensors

Monitoring tools

SNMP-based NMS: Nagios, Zabbix, PRTG, SolarWinds; NetFlow for traffic; **DCIM** / BMS for power, cooling and space; SIEM for security logs; all watched 24/7 from a **NOC**.

Server hardware & buying checklist: multi-core **CPUs** (1–2 sockets), **ECC RAM** (corrects bit errors), **RAID controller** with hot-swap drives, redundant hot-swap **PSUs** and fans, multiple NICs (10/25 GbE), remote management **BMC** (iLO, iDRAC, IPMI), BIOS/UEFI settings (boot order, RAID, virtualisation on). **Before buying**: workload sizing and growth, form factor (rack/blade), warranty and on-site support SLA, compatibility with the hypervisor and OS, power draw, vendor track record, total cost of ownership.
SAS vs SATA: SAS is full-duplex, dual-ported, built for 24/7 duty → **servers**; SATA is cheaper and larger, for desktops and cold storage. SSDs are preferred for low latency and no moving parts.

Memory trick: PUE: "**P**ower in the building ÷ **U**seful (IT) power" → closer to 1 is better. Server reliability = "**E**CC, **R**AID, **R**edundant PSU": **ERR** on the side of safety.

11 Disaster Recovery, BCP, RTO & RPO

DR vs BCP: A **disaster recovery plan** restores **IT systems and data** after fire, flood, earthquake, ransomware or power loss. A **business continuity plan** keeps the **whole business** running (people, branches, suppliers); DR is the IT part of BCP. A **DRS** is a second site, far from the primary **DC**, holding replicated systems.



- **RPO** (Recovery Point Objective): the **maximum data loss** allowed, measured backwards in time. RPO 15 min → replicate at least every 15 min
- **RTO** (Recovery Time Objective): the **maximum downtime** allowed until service is back
- **MTBF** (mean time between failures), **MTTR** (mean time to repair): availability = MTBF ÷ (MTBF + MTTR)

Site	State	RTO	Cost
Hot	live replica, synced	minutes	highest
Warm	hardware ready, data some hours old	hours	medium
Cold	space and power only	days	lowest
DRaaS	cloud replica	min–hours	pay per use

Scenario: DC in Dhaka, DR in Khulna: ① business impact analysis: set RTO/RPO per system (core banking, ATM switch, mail) ② pick the DR site in a **different seismic and flood zone** on a different power grid ③ two diverse links from different carriers (fibre + backup) ④ replicate: synchronous for near-zero RPO only over short distance; **asynchronous** over ~300 km Dhaka–Khulna ⑤ compatible hardware, licences, DNS/IP failover ⑥ backups follow **3-2-1** (3 copies, 2 media, 1 off site, plus one offline/immutable against ransomware) ⑦ runbook, roles, call tree ⑧ DR drill twice a year, then fail back.

DC vs DRS: DC = primary, serves live traffic; DRS = standby copy that takes over on disaster (active-passive, or active-active sharing load). **Colocation** DR = renting racks, power and cooling in a provider's DC: saves building cost, but no **granular climate control**.

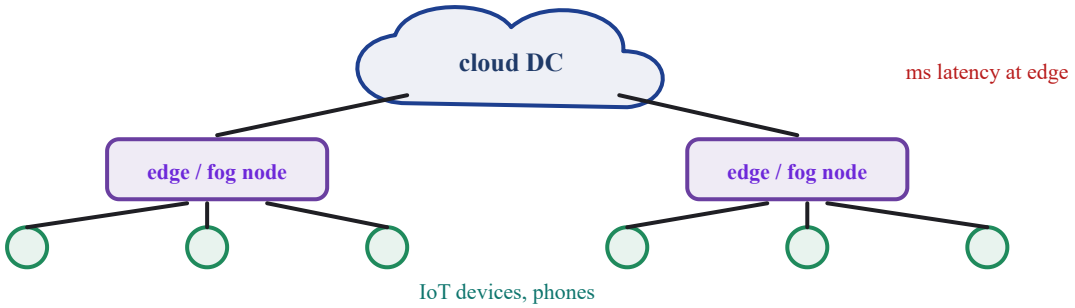
Maintenance

Preventive: planned before failure (filters, battery tests, patches, generator load test) · **Corrective**: repair after failure (replace a failed disk, PSU). Good practice: cleaning, monitoring, patch windows, backup checks, recorded BIOS settings: longer life, fewer outages.

Memory trick: "**R**PO = **R**ewind **P**oint (data); **R**TO = **R**estart **T**ime." Hot-warm-cold = minutes-hours-days.

12 Edge, Fog & CDN

Edge computing: processes data **near where it is produced** (on the device, a factory gateway, a mobile base station) instead of sending everything to a distant cloud. **Fog computing** (Cisco, 2012) is the layer of local nodes between devices and the cloud. An **edge server** is a server placed at the network edge, close to users.



Why an edge server?: ① **low latency** (self-driving cars, AR, online games, factory control) ② saves backbone **bandwidth** (filter video/IoT data locally) ③ keeps working when the WAN link is down ④ data stays local for privacy rules ⑤ serves cached content near users. Used by 5G MEC, smart meters, CCTV analytics.

CDN (content delivery network): a network of edge cache servers that serve static content (images, video, scripts) from the nearest location: Akamai, Cloudflare, Amazon CloudFront. Cuts latency and absorbs DDoS.

Cloud vs edge: the cloud is centralised, huge and elastic, good for heavy analytics and storage; the edge is distributed, small and close, good for real-time decisions. They work together.

13 Big Data & Hadoop (cloud workloads)

Big data: data sets too large, fast or varied for a traditional database to store and process. The **5 V's**: **Volume**, **Velocity**, **Variety**, **Veracity**, **Value**. Examples: bank transactions and server logs, mobile call records, social-media posts, sensor and CCTV streams.

Hadoop component	Role (fraud detection on server logs)
HDFS	distributed, replicated (3 copies) file storage for the raw logs
YARN	resource manager: schedules jobs on the cluster
MapReduce	batch processing: map (key-value pairs) → shuffle → reduce
Kafka + **Spark Streaming**	ingest log events and score them in near real time
Hive / HBase, **Spark MLlib**	SQL queries, fast row lookups, ML models for predictive maintenance

Memory trick: Edge = "**process at the doorstep**"; cloud = "**process at head office**". Big data 5 V's: "**V**ery **V**ast **V**aried **V**erified **V**aluable".

Exam trap: Edge computing does **not** replace the cloud; fog sits between edge and cloud. MapReduce is batch, not real-time; real-time needs a stream engine (Spark Streaming, Flink).

14 Moving a Data Centre to the Cloud

Cloud migration: moves applications, data and infrastructure from an on-premises data centre to a cloud. It is planned app by app, using the **"R" strategies** (Gartner 5 R's, 2010; AWS later lists 7).

Strategy	What it means	Bank example
Rehost (lift & shift)	move VMs as they are to IaaS	file and print servers
Replatform	small changes, e.g. move to a managed DB	intranet DB → managed SQL
Refactor	re-architect as cloud-native microservices	mobile banking API
Repurchase	switch to a SaaS product	email → Microsoft 365
Retire / Retain (+ Relocate)	switch off unused apps / keep on-prem for now	core banking kept in-country



regulator approval and security review run through every phase

Model answer: "transform the bank's data centre into a cloud": ① **Assess**: list every application, its data class, dependencies and current cost; set RTO/RPO ② **Decide the model**: hybrid; customer and core-banking data stay in a private cloud or an in-country provider, as the BB cloud guideline and s.12 of the Bank Company Act require; public cloud for web, analytics, DR, dev/test ③ **Choose a provider**: Tier III+ facility, ISO 27001, PCI DSS, local presence, SLA (99.95%+), exit clause ④ **Build a landing zone**: accounts, network (VPN/direct link), IAM with MFA, encryption keys, logging ⑤ **Migrate in waves**, low-risk first, parallel run, rollback plan ⑥ **Secure & govern**: shared-responsibility matrix, CSPM tools, audits, staff training ⑦ **Optimise**: autoscaling, reserved instances, cost reports.

Scenario: 10k → 50k hits, no cloud: **Technical**: CPU/RAM saturate, slow pages, time-outs and HTTP 5xx errors (500 internal error, 503 unavailable), DB lock contention, no headroom for a DDoS. **Business**: failed transactions, lost customers and trust, emergency CapEx for servers that sit idle after the peak, staff overtime, possible regulatory penalty. Fix: load balancer + horizontal scaling, caching/CDN, or hybrid cloud bursting.

Cloud outage: who is responsible?: **Technically** the provider answers for its infrastructure under the **SLA** (service credits); the bank answers for its architecture (single region? no DR?). **Legally** the bank stays accountable to customers and Bangladesh Bank: outsourcing a service never outsources the responsibility. Hence: multi-zone design, DR plan, incident reporting, contract penalties.

Memory trick: R's: "**Re-host, Re-platform, Re-factor, Re-purchase, Retire, Retain**": from "move as is" to "rewrite" to "drop". Phases: "**A P P M O**" = Assess, Plan, Pilot, Migrate, Operate.

Night-Before-the-Exam Sheet

Fact sheet:

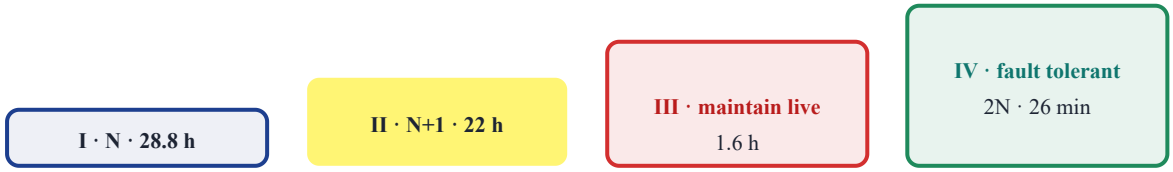
NIST SP 800-145 (2011): 5 · 3 · 4	5 traits: O-B-R-R-M
IaaS = VMs · PaaS = deploy code	SaaS = finished app (Gmail, M365)
public · private · community · hybrid	hybrid → cloud bursting
Type-1: ESXi, Hyper-V, Xen, KVM	Type-2: VirtualBox, Workstation
VM = own OS · container = shared kernel	Docker 2013 · Kubernetes 2014
vertical = scale up · horizontal = out	max VMs = min over CPU, RAM, disk
Tier III 99.982% (1.6 h)	Tier IV 99.995% (26.3 min)
N+1 = one spare · 2N = full copy	UPS 0 ms (online) · IPS delayed
PUE = total ÷ IT power (avg 1.56)	ASHRAE inlet ≈ 18–27 °C
RPO = data loss · RTO = downtime	hot = min · warm = h · cold = days
series $R = R_1R_2R_3 \cdot 0.95^3 = 0.857$	3-2-1 backup + 1 offline
BB cloud guideline: 16 Mar 2023	Kaliakoir Tier IV DC: 28 Nov 2019

Top 14 one-liners for MCQs

1. Pooling/sharing concept = virtualisation
2. Cloud services are usually stateless
3. CaaS (computing) is not a NIST layer
4. Google App Engine, Heroku = PaaS
5. AWS EC2 = IaaS; Salesforce = SaaS
6. JVM is a process VM, not a hypervisor
7. Docker Hub = cloud image registry
8. K8s smallest unit = Pod; state in etcd
9. Tier III = concurrently maintainable
10. Tier IV = fault tolerant
11. Servers prefer SAS/SSD and ECC RAM
12. Fire suppression = clean gas, not water
13. Colocation: no granular climate control
14. HTTP 500 = internal server error

Top traps: private = exclusive use, not "on site" · SaaS customer still owns data and access · Type-1 is faster than Type-2 · containers isolate less than VMs · tiers rate power/cooling paths, not server count · UPS bridges minutes, the generator carries hours · RPO is about data, RTO about time · MapReduce is batch · outsourcing to cloud never outsources responsibility

All tricks together: O-B-R-R-M · pizza as a service · Park / home / Club / Home + park · provider guards the house · Type-1 on metal · VM brings its OS · taller vs wider · 29-22-1.6-0.4 h · UPS unbroken · ERR (ECC, RAID, Redundant PSU) · Rewind Point, Restart Time · doorstep vs head office · A-P-P-M-O



✓ Practice MCQs (exam pattern, 10 marks)

1. A developer only wants to deploy code; the model is:
(a) IaaS (b) SaaS
(c) PaaS (d) on-premises

2. Not a cloud service layer:
(a) CaaS (Computing) (b) IaaS
(c) PaaS (d) SaaS

3. A Type-2 (hosted) hypervisor:
(a) VMware ESXi (b) Xen
(c) Hyper-V (d) VirtualBox

4. Cloud concept for pooling and sharing resources:
(a) polymorphism (b) virtualisation
(c) abstraction (d) digitising

5. Concurrently maintainable data centre tier:
(a) Tier III (b) Tier I
(c) Tier II (d) Tier IV

6. Maximum acceptable data loss is the:
(a) RTO (b) MTTR
(c) RPO (d) SLA

7. Docker Hub is:
(a) a hypervisor (b) a database
(c) a language (d) a cloud image registry

8. 32 cores, 96 GB, 4 TB host; VM 4 c / 16 GB / 500 GB. Max VMs:
(a) 8 (b) 6
(c) 4 (d) 24

9. Not a usual reason for colocation-based DR:
(a) full hardware control (b) security, location
(c) cost of building (d) granular climate control

10. Total 1,800 kW, IT load 1,200 kW. PUE:
(a) 0.67 (b) 1.2
(c) 1.5 (d) 1.8

Answer key → 1-c · 2-a · 3-d · 4-b · 5-a · 6-c · 7-d · 8-b · 9-d · 10-c

Written question: Define a virtual machine and a hypervisor with a diagram. Distinguish Type-1 and Type-2 hypervisors and a VM from a container. List four benefits of VMs. 10 marks

1. **Definitions**: VM = software computer with its own guest OS; hypervisor = layer that creates, isolates and schedules VMs on real hardware 2
2. **Diagram**: hardware → hypervisor → VMs (Type-1); hardware → host OS → hypervisor → VMs (Type-2) 2
3. **Type-1 vs Type-2**: bare metal, faster (ESXi, Hyper-V, KVM) vs hosted (VirtualBox, Workstation) 2
4. **VM vs container**: full OS, GBs, minutes, strong isolation vs shared kernel, MBs, seconds 2
5. **Benefits**: consolidation, isolation, snapshots, live migration, easy DR 2

Before the exam ✓: IaaS/PaaS/SaaS stack drawn · Type-1 vs Type-2 diagram · tier table · RTO/RPO timeline. **Also practise (written)**: cloud traits, pros and cons · DR plan Dhaka → Khulna · DC → cloud migration plan · UPS vs IPS.